NATIONAL UNIFIED ELECTRONIC RECEIPT SYSTEM

DIGITAL TRANSACTIONS. A STRONGER PHILIPPINES.

FOR MERCHANTS & THEIR INTEGRATION DEVELOPERS

Merchant API Connection Guide

Connect your ERP, POS or ecommerce system to your NUERS account.

One connection. Reliable transaction records.

01 SET UP

Choose Live or Sandbox. Protect your API key.

02 CONNECT

Submit original transactions with clear tax mapping.

03 RECONCILE

Recover offline events and verify signed callbacks.

EDITION 1.5 | 29 OCTOBER 2026

Contract: NUERS-ingestion-1 | HTTPS / JSON | PHP currency

START HERE

Your route to a working connection

This guide is for a merchant and the developer connecting the merchant's own system. Follow the setup once, implement the payload contract, then test recovery before going Live.

Pages	Connection topic
3-6	Prepare your account, credentials and environment
7-10	Endpoints, fields, items and document types
11-15	Complete payloads, tax mapping and special sales
16-19	cURL, Node.js and PHP connection examples
20-23	Acceptance, status, wallet and batch processing
24-26	Offline outbox, duplicate protection and manual retry
27-31	Webhook configuration, verification and delivery
32-34	Errors, security and go-live checklist
35-38	Reconciliation, quick reference and manual discounts
39-42	Sales status, customer email, branding and checklist
43-46	Buyer CWT, corrections, manual recovery and fee invoices
47-48	Invoice numbering, buyer identity and printed registration

Use this guide in order

Merchant owner: start with pages 3-6, 22, 26 and 34.
Integration developer: read the field contract and examples, then implement the outbox and response checks.
Webhook developer: pages 27-31 are an optional completion channel; status polling remains available.

> Examples are synthetic
> Replace sample connection identifiers with your own dashboard values. Use a Sandbox key for sample sales. The dates and document numbers printed here are examples, not real transactions.

PDF navigation: click a contents page number or use the chapter bookmarks. Literal field names and enum values are case-sensitive. Amounts in transaction JSON are PHP pesos; wallet fields ending in _centavos are integer centavos.

01 / SET UP

From verified account to API key

01 Sign in as the registered business owner / Merchant Admin and complete the login verification. Open Business Profile and confirm that the business is Active and its taxpayer details are correct.

02 Open Merchant > API Integration. Choose Generate New API Key, enter a clear name such as ERP Connector, and choose Sandbox for initial testing.

03 Copy the API key when it is shown. Store it in your server's secret manager. It is shown once; the portal does not reveal the original secret again.

04 Copy the assigned source_profile_id, source_system and branch_code from the connection / ERP configuration or Source Profiles tab.

05 Use the wallet endpoint on page 7 to check authentication and confirm the environment and assigned business context. Then submit a synthetic Sandbox event.

06 After completing the checklist on page 34, generate a separate Live key and use the Live connection configuration for actual transactions.

> Automatic business connection
> An active, verified merchant receives its owned-system connection automatically. There is no second source approval and no merchant Branches / Terminals form to complete before creating this connection.

If the button is disabled: enter a key name, check that you are the registered owner, then use Refresh Status or reload once. An already-active account does not need another approval. If it remains blocked, send a redacted screenshot and time to Submit a Request. Never include the secret key.

01 / SET UP

Copy the connection configuration

Setting	Where to get it / how to use it
API base URL	https://nuers.net/api/integrations/transactions
API key	One-time secret generated in API Integration. Send in X-NUERS-API-Key.
Environment	The key's fixed Live or Sandbox environment. Keep separate configuration and queues.
source_profile_id	Assigned UUID shown in Source Profiles / ERP configuration. Required for source_capture.
source_system	Exact assigned identifier. Do not replace it with a label such as SAP or My ERP.
branch_code	Assigned five-digit string, including leading zeros. Use the same value as the key.
merchant_id	Returned by the authenticated wallet endpoint. Useful for verifying signed webhook events.

Suggested server configuration

```
NUERS_API_BASE=https://nuers.net/api/integrations/transactions
NUERS_ENVIRONMENT=sandbox
NUERS_API_KEY=<load from your secret manager>
NUERS_SOURCE_PROFILE_ID=<copy assigned UUID>
NUERS_SOURCE_SYSTEM=<copy assigned identifier>
NUERS_BRANCH_CODE=<copy all five digits>
```

The values above are connector configuration names, not additional API fields. Keep them outside public web folders and source control. Multiple owned systems can share the assigned source context; prefix their event IDs and external references, for example erpA:invoice:0001 and posB:invoice:0001.

> Use source_capture for an existing original
> Always send record_mode: source_capture when your ERP or POS has already issued the document. Omitting the field defaults to the separate NUERS issuance workflow and can fail authority checks.

01 / SET UP

Keep Sandbox and Live separate

Behavior	Sandbox	Live
API URL	Same HTTPS API base	Same HTTPS API base
Authentication	Sandbox key	Live key
Records	Isolated test records	Merchant's actual financial records
Wallet	No debit; simulated fee information	Eligible fee debited atomically
Identifiers	Test transaction / receipt prefixes	Live record identifiers
BIR acceptance	not_applicable	Inspect the response; intake alone is not BIR acceptance
Purpose	Synthetic integration and retry tests	Actual finalized business events

Environment rules for your connector

01 Use separate secret-store entries, outbox queues, response stores and webhook configuration for each environment. The environment belongs to the API key; a JSON flag cannot switch it.

02 Check the response environment before marking any event delivered. A Sandbox success must never clear a pending Live event.

03 Keep test buyers, document numbers and reports out of Live. Select the matching environment when viewing merchant records and API logs.

> Sandbox is a connector test
> Sandbox validates payloads against the merchant's current tax classification. It does not prove that a Live source profile covers the original date or historical seller identity. Check the assigned Live profile and an authorized real pilot before cutover.

01 / SET UP

Authentication and credential care

```
X-NUERS-API-Key: <your server-held API key>
Content-Type: application/json
Accept: application/json
```

Use the header above on every API call. The API key is itself the secret credential; a separate public key, private key pair or token exchange is not required for this connection.

Control	Merchant action
Scopes	transactions:write submits events. transactions:read reads status and wallet. Manual retry needs both.
IP restrictions	Use Edit Key Policy to allow the connector's exact public IPv4 / IPv6 egress addresses. Include your failover egress before switching.
Expiry	Generated keys expire after 90 days. Track the displayed expiry and rotate before it.
Daily quota	Default Live key: 50,000 requests/day. Sandbox: 5,000/day. Check the key's current policy. Other shared limits also apply.
Owner access	Credential changes require the registered owner's verified login. Keep MFA enabled.
Lost key	Generate a replacement. The original secret cannot be recovered from the masked display.

Safe rotation

Create the replacement in the same environment and business context. Update the server secret, verify wallet/status access, then revoke the old key. Keep the original event IDs and JSON when retrying queued events. Revocation is permanent.

> Secrets stay on the server
> Do not embed API keys in browser code, mobile apps, URLs, screenshots, support tickets, webhook bodies or logs. Keep HTTPS certificate verification enabled and redact authentication headers.

02 / API REFERENCE

Four endpoints for your connector

Base URL
https://nuers.net/api/integrations/transactions

Method / path	Purpose	Scope
POST /	Store one finalized transaction copy.	transactions:write
POST /batch	Store 1-50 events atomically.	transactions:write
GET /{id}	Read the accepted intake record by its returned id.	transactions:read
GET /wallet	Read environment, connection context, balance and pricing.	transactions:read

Paths above are relative to the base URL. POST / means POST to the base URL itself. For example, the wallet URL is:

```
https://nuers.net/api/integrations/transactions/wallet
```

First connectivity check

```
curl --silent --show-error --fail-with-body \
  "https://nuers.net/api/integrations/transactions/wallet" \
  --header "X-NUERS-API-Key: ${NUERS_API_KEY}" \
  --header "Accept: application/json"
```

Load NUERS_API_KEY privately before running the command. Confirm HTTP 200, the expected environment and the assigned branch_code. A successful wallet check confirms authentication and read scope; it does not create a transaction or guarantee a later write will pass.

> Use the intake id for status
> GET /{id} uses the UUID named id from the acceptance response. It does not use receipt_id, source_event_id or receipt_number. The key must belong to the same merchant, branch and environment.

02 / API REFERENCE

Required event fields

Field	Required value / rule
schema_version	Exactly NUERS-ingestion-1.
record_mode	Send source_capture explicitly for copies from your system.
source_profile_id	Assigned source UUID; required with source_capture.
source_system	Exact assigned identifier. Maximum 120 characters; letters, digits, underscore, dot and hyphen.
source_event_id	Stable original event identifier, maximum 120 characters. Never regenerate it on a retry.
external_reference	Stable source business reference, maximum 120 characters. Namespace it by system.
event_type	invoice.finalized for a sales invoice; payment.recorded for a supplementary payment receipt.
occurred_at	Original ISO 8601 timestamp with Z or an offset, e.g. +08:00. At most five minutes in the future.
branch_code	Exact assigned five-digit string; preserve leading zeros.
receipt_number	Original seller document number, 1-50 characters. Preserve prefix and leading zeros.
document_type	Use sales_invoice to classify from calculated lines, or an explicit type from page 10.
payment_method	cash, card, bank_transfer, e_wallet, credit or other.
gross_amount	Final PHP total after discounts and VAT, before withholding. Prefer a two-decimal string.
tax_type	Default line treatment: vatable, vat_exempt, zero_rated or non_vat. Do not send mixed here.
items	An array of 1-100 line items. See page 9.

Unknown fields are rejected. Use the names shown here rather than sending an entire ERP object. Send vat_inclusive explicitly even though its default is true.

02 / API REFERENCE

Line items and optional details

Line field	Rule
description	Required text, up to 1,000 characters.
quantity	Required; greater than zero, at most 1,000,000; up to four decimal places.
unit_price	Required PHP amount; zero or greater, at most 100,000,000; up to two decimals.
tax_type	Defaults to header tax_type. Set it explicitly on each line in a mixed invoice.
vat_rate	Optional: 12 for vatable; 0 for other treatments.
discount_pct / manual_discount	Legacy percentage or amount/percentage object. See pages 37-38. Do not stack discounts.
line_total	Optional asserted final line total. If supplied, it must equal the server calculation.
tax_basis_reference	Optional evidence reference, up to 2,000 characters. Include the basis for exempt or zero-rated treatment.
benefit_eligible	Boolean. Set true only on lines qualifying for the declared sales_benefit.

Optional event details

customer_name (255), customer_address (2,000), and customer_tin may accompany the original. A TIN must be 9, 12 or 14 digits, or a 3-3-3 formatted TIN with an optional 3- or 5-digit suffix. Keep it as text.

tax_basis_reference at event level is inherited by its lines. withholding_tax requires withholding_basis_reference (3-2,000 characters). sales_benefit is described on pages 14-15.

> Use clean numeric values
> Send "112.00", not "PHP 112.00" or "1,000.00". Booleans should be JSON true / false. Omit optional fields you do not use; once accepted, preserve the original JSON types and field presence on retries.

02 / API REFERENCE

Document types and tax treatments

document_type	event_type / meaning
sales_invoice	invoice.finalized. Recommended: server classifies from the calculated lines.
vat_invoice	invoice.finalized; all lines vatable, VAT seller.
non_vat_invoice	invoice.finalized; ordinary non-VAT or wholly exempt sale by a non-VAT seller.
vat_exempt_invoice	invoice.finalized; all lines exempt, VAT seller.
zero_rated_invoice	invoice.finalized; all lines zero-rated, VAT seller.
mixed_sales_invoice	invoice.finalized; more than one permitted line treatment.
payment_receipt	payment.recorded; supplementary payment evidence.
official_receipt	Compatibility alias for payment_receipt. Use payment.recorded; it is not the primary sales invoice.

Seller registration controls line treatment

Seller	Allowed input line tax_type
VAT registered	vatable, vat_exempt, zero_rated
Non-VAT registered	non_vat, vat_exempt

A VAT merchant can mix taxable, exempt and zero-rated products when the classification is supported by its evidence. Use vat_exempt for an exempt product; non_vat is the seller's non-VAT treatment. Do not change the merchant's registration just to submit one item.

Payment receipts must use their own stable event identity. Keep the relationship to the original invoice in your source records. Arbitrary cancellation, credit-note and debit-note types are not accepted by this contract.

03 / PAYLOAD EXAMPLES

Complete non-VAT invoice copy

Valid shape for a non-VAT merchant. Replace the sample connection values, original IDs and timestamp. Use this structure as event.json for the sender examples.

```
{
  "schema_version": "NUERS-ingestion-1",
  "record_mode": "source_capture",
  "source_profile_id": "00000000-0000-4000-8000-000000000001",
  "source_system": "SAMPLE-owned-system",
  "source_event_id": "erpA:invoice:0001",
  "external_reference": "erpA:INV-0001",
  "event_type": "invoice.finalized",
  "occurred_at": "2026-10-18T10:00:00+08:00",
  "branch_code": "00000",
  "receipt_number": "SAMPLE-INV-0001",
  "document_type": "sales_invoice",
  "payment_method": "cash",
  "gross_amount": "100.00",
  "tax_type": "non_vat",
  "vat_inclusive": true,
  "items": [
    {
      "description": "SAMPLE product",
      "quantity": 1,
      "unit_price": "100.00",
      "tax_type": "non_vat"
    }
  ]
}
```

Expected calculation
PHP 100.00 gross, PHP 0.00 VAT and PHP 100.00 ordinary non-VAT sales (sspt_sales). The API does not calculate a seller's percentage-tax liability from this field.

03 / PAYLOAD EXAMPLES

VAT and mixed-product invoices

Start with the full event on page 11. For a VAT merchant, replace the following fields. Keep all connection and original-document fields from that complete event.

```
{
  "document_type": "sales_invoice",
  "gross_amount": "187.00",
  "tax_type": "vatable",
  "vat_inclusive": true,
  "tax_basis_reference": "SAMPLE approved product tax mapping",
  "items": [
    {
      "description": "SAMPLE taxable product",
      "quantity": 1,
      "unit_price": "112.00",
      "tax_type": "vatable"
    },
    {
      "description": "SAMPLE exempt product",
      "quantity": 1,
      "unit_price": "50.00",
      "tax_type": "vat_exempt"
    },
    {
      "description": "SAMPLE zero-rated product",
      "quantity": 1,
      "unit_price": "25.00",
      "tax_type": "zero_rated"
    }
  ]
}
```

Calculated bucket	PHP
VATable base	100.00
VAT	12.00
VAT-exempt sales	50.00
Zero-rated sales	25.00
gross_amount	187.00

This is a patch illustration, not a complete standalone request. Replace the sample tax-basis reference with the source evidence supporting your product classification. The result resolves to mixed_sales_invoice.

03 / PAYLOAD EXAMPLES

Prices, discounts and withholding

Input	Expected final amounts
Inclusive price: 1 x PHP 112.00, vatable	Base 100.00 + VAT 12.00 = gross 112.00.
Exclusive price: 1 x PHP 100.00, vatable; vat_inclusive false	Base 100.00 + VAT 12.00 = gross 112.00.
Inclusive: 2 x PHP 112.00; discount_pct 10	Ordinary discount 22.40; base 180.00 + VAT 21.60 = gross 201.60.
Gross 112.00; withholding_tax 2.00	gross_amount remains 112.00. net_payable is 110.00. Include withholding_basis_reference.

Reconciliation rules

01 Choose one price basis for the whole document with vat_inclusive. Express unit_price in that basis.

02 Calculate each line using quantity, price and applicable discount; round currency amounts to centavos. Calculate each line's VAT, then sum the lines.

03 Send gross_amount as the final total including any VAT and after discounts, before withholding. The server recalculates and rejects a mismatch.

04 Compare the returned calculation to your original source document. Do not silently alter an already-issued original to force a match.

> Do not pre-discount twice
> For a statutory benefit, supply the original pre-benefit unit_price and mark eligible lines. NUERS applies the category calculation. A pre-discounted price plus benefit_eligible would apply the benefit again.

Use integer centavos or a decimal library in your own accounting code. Binary floating-point addition should not determine monetary equality. The accepted document total is capped at PHP 10 billion.

03 / SPECIAL SALES

Senior, PWD, athlete and solo parent

Send sales_benefit only when the merchant has checked the applicable eligibility evidence. The API records the category and calculates the declared benefit; a buyer label alone does not establish entitlement.

category	Accepted scope values
senior_citizen or pwd	restaurant, hotel, recreation, admission, medicine, medical_dental, domestic_transport, funeral
national_athlete_coach	restaurant, hotel, recreation, admission, medicine, sports_equipment, domestic_transport
solo_parent	baby_milk, baby_food, micronutrient_supplements, diapers, medicine, vaccines, medical_supplements

Required sales_benefit fields

Field	Rule
category / scope	Use one supported category and a matching scope from the table.
beneficiary_name	2-200 characters.
id_number	2-100 characters.
evidence_reference	5-500 characters; a private source record reference.
eligibility_confirmed	true only after the merchant checks the evidence.

Use document_type: sales_invoice and mark only eligible lines with benefit_eligible: true. One benefit category applies per document. Separate non-eligible lines and avoid combining different beneficiary categories in one event.

> Additional solo-parent fields
> Provide child_birth_date, id_valid_until, income_below_250000: true and booklet_reference. Dates use YYYY-MM-DD. The API checks that the child is at most six years old on the original transaction date and that the identification is unexpired. For other categories, id_valid_until is optional but checked when supplied.

These are API validation rules. Your merchant tax/compliance reviewer must establish the covered purchase and retain its underlying evidence. Do not send image attachments or full identity documents in the transaction JSON.

03 / SPECIAL SALES

Benefit example and expected totals

Apply this patch to the complete event on page 11 for an eligible VAT-registered seller's PWD purchase. It replaces the tax, amount and items fields; other required event fields remain.

```
{
  "document_type": "sales_invoice",
  "gross_amount": "800.00",
  "tax_type": "vatable",
  "vat_inclusive": true,
  "sales_benefit": {
    "category": "pwd",
    "scope": "restaurant",
    "beneficiary_name": "SAMPLE Customer",
    "id_number": "SAMPLE-ID-001",
    "evidence_reference": "SAMPLE verified eligibility record",
    "eligibility_confirmed": true
  },
  "items": [
    {
      "description": "SAMPLE eligible purchase",
      "quantity": 1,
      "unit_price": "1120.00",
      "tax_type": "vatable",
      "benefit_eligible": true
    }
  ]
}
```

For the same eligible PHP 1,120.00 VAT-inclusive pre-benefit line, the current calculator produces:

Category	Gross	Discount	VAT	VAT relief
Senior / PWD	800.00	200.00	0.00	120.00
Athlete / coach	896.00	200.00	96.00	24.00
Solo parent	900.00	100.00	0.00	120.00

Amounts above are PHP and apply only to this qualifying example. Ordinary discount_pct cannot be stacked on an eligible statutory-benefit line. Returned sale_category, statutory_discount_amount and vat_relief_amount support reconciliation and filtered reports.

04 / SEND REQUESTS

cURL and Postman quick start

Submit one saved JSON file

```
curl --silent --show-error \
  --connect-timeout 10 --max-time 45 \
  --request POST \
  "https://nuers.net/api/integrations/transactions" \
  --header "X-NUERS-API-Key: ${NUERS_API_KEY}" \
  --header "Content-Type: application/json" \
  --header "Accept: application/json" \
  --data-binary @event.json \
  --output response.json \
  --write-out 'HTTP %{http_code}\n'
```

event.json must contain a complete event. Save its exact content in your outbox first. A network timeout leaves the result unknown; keep the event pending and retry unchanged. Inspect response.json and the HTTP status together.

Postman setup

01 Create separate private Sandbox and Live environments. Set base_url to the API base and api_key as a secret variable. Do not export or share its current value.

02 Create a POST request to {{base_url}}. Add X-NUERS-API-Key: {{api_key}}, Content-Type: application/json and Accept: application/json.

03 Choose Body > raw > JSON. Paste the complete event with your assigned identifiers, then send in Sandbox.

04 Verify HTTP 201 and status: durably_recorded. Save id. Repeat the same request and expect HTTP 200 with duplicate: true and the same IDs.

> Development tool, not the delivery queue
> Postman and cURL help inspect a request. Your production connector still needs durable storage, controlled retries and response reconciliation. Avoid verbose command output that exposes authentication headers.

04 / SENDER CODE

Node.js: one controlled send

Node.js 20+ example. The caller supplies the exact JSON already saved in its durable outbox. This function performs one attempt and returns an acknowledgement only after checking identity and environment.

```
export async function sendNuers(savedJson) {
  const event = JSON.parse(savedJson);
  const key = process.env.NUERS_API_KEY;
  const env = process.env.NUERS_ENVIRONMENT;
  if (!key || !['live', 'sandbox'].includes(env)) {
    throw new Error('Missing private connector configuration');
  }
  const url =
    'https://nuers.net/api/integrations/transactions';
  const res = await fetch(url, {
    method: 'POST', redirect: 'error',
    signal: AbortSignal.timeout(45000),
    headers: {
      'X-NUERS-API-Key': key,
      'Content-Type': 'application/json',
      'Accept': 'application/json'
    },
    body: savedJson
  });
  const body = await res.json().catch(() => null);
  const accepted = res.status === 201 ||
    (res.status === 200 && body?.duplicate === true);
  const valid = accepted && body?.id && body?.receipt_id &&
    body?.transaction_id && body.environment === env &&
    body.status === 'durably_recorded' &&
    body.nuers_status === 'success' &&
    body.source_event_id === event.source_event_id &&
    body.external_reference === event.external_reference;
  if (!valid) {
    const err = new Error('NUERS response needs review/retry');
    err.status = res.status;
    err.retryAfter = res.headers.get('retry-after');
    // Redact personal data before logging field-level errors.
    throw err;
  }
  return body; // Caller commits IDs + delivered state locally.
}
```

A thrown network/parse error must leave the outbox pending. Handle HTTP statuses using page 32. Store the returned IDs and delivered state atomically. Do not print savedJson or the API key.

04 / SENDER CODE

PHP: request and transport handling

PHP 8.2+ with cURL. This is the first half of a single function; continue directly with page 19. It sends the JSON string already stored in your outbox.

```php
<?php
function sendNuers(string $savedJson): array
{
    $event = json_decode($savedJson, true, 64,
        JSON_THROW_ON_ERROR);
    $key = getenv('NUERS_API_KEY');
    $env = getenv('NUERS_ENVIRONMENT');
    if (!$key || !in_array($env, ['live', 'sandbox'], true)) {
        throw new RuntimeException('Missing connector config');
    }
    $retryAfter = null;
    $ch = curl_init(
        'https://nuers.net/api/integrations/transactions'
    );
    curl_setopt_array($ch, [
        CURLOPT_POST => true,
        CURLOPT_POSTFIELDS => $savedJson,
        CURLOPT_RETURNTRANSFER => true,
        CURLOPT_CONNECTTIMEOUT => 10,
        CURLOPT_TIMEOUT => 45,
        CURLOPT_FOLLOWLOCATION => false,
        CURLOPT_SSL_VERIFYPEER => true,
        CURLOPT_SSL_VERIFYHOST => 2,
        CURLOPT_HTTPHEADER => [
            'X-NUERS-API-Key: '.$key,
            'Content-Type: application/json',
            'Accept: application/json',
        ],
        CURLOPT_HEADERFUNCTION =>
            function ($ch, $line) use (&$retryAfter) {
                if (stripos($line, 'Retry-After:') === 0) {
                    $retryAfter = trim(substr($line, 12));
                }
                return strlen($line);
            },
    ]);
    $raw = curl_exec($ch);
    $status = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
    curl_close($ch);
    if ($raw === false) {
        throw new RuntimeException('Transport outcome unknown');
    }
```

04 / SENDER CODE

PHP: validate the acknowledgement

```
    $body = json_decode($raw, true, 64, JSON_THROW_ON_ERROR);
    $accepted = $status === 201 ||
        ($status === 200 && ($body['duplicate'] ?? false) === true);
    $valid = $accepted && !empty($body['id']) &&
        !empty($body['transaction_id']) &&
        !empty($body['receipt_id']) &&
        ($body['environment'] ?? null) === $env &&
        ($body['status'] ?? null) === 'durably_recorded' &&
        ($body['nuers_status'] ?? null) === 'success' &&
        ($body['source_event_id'] ?? null) ===
            $event['source_event_id'] &&
        ($body['external_reference'] ?? null) ===
            $event['external_reference'];
    if (!$valid) {
        return ['acknowledged' => false,
            'http_status' => $status,
            'retry_after' => $retryAfter];
    }
    return ['acknowledged' => true, 'response' => $body];
}
```

How to connect the function to your worker

01 Load one pending event from durable storage and take a worker lock so two local workers do not process it concurrently.

02 Call sendNuers with the stored original JSON. A thrown exception or acknowledged: false leaves it pending or marks it for operator review according to page 32.

03 If acknowledged is true, commit response.id, transaction_id, receipt_id, receipt_number and the delivered state in one local database transaction.

04 Release the worker lock. If the local commit fails, leave the event pending. A later unchanged retry should return the already-accepted IDs.

> Complete the operational layer
> These examples show authenticated sending and acknowledgement checks. Your application supplies persistent queues, database transactions, scheduling, alerting and access control. Do not use an in-memory loop as the only record of unsent sales.

05 / RESPONSES

Understand a successful acceptance

A newly accepted single event returns HTTP 201. A previously accepted identical event returns HTTP 200 with duplicate: true. The following is an abbreviated Live response; calculation and wallet contain more detail.


```
{
  "duplicate": false,
  "environment": "live",
  "id": "11111111-1111-4111-8111-111111111111",
  "transaction_id": "22222222-2222-4222-8222-222222222222",
  "receipt_id": "33333333-3333-4333-8333-333333333333",
  "receipt_number": "SAMPLE-INV-0001",
  "external_reference": "erpA:INV-0001",
  "record_mode": "source_capture",
  "source_event_id": "erpA:invoice:0001",
  "occurred_at": "2026-10-18T10:00:00+08:00",
  "accepted_at": "2026-10-18T10:00:03+08:00",
  "status": "durably_recorded",
  "nuers_status": "success"
}
```


Check	What your connector must do
Status + identity	Require the expected environment, durably_recorded, non-empty IDs and matching event / external reference.
Local commit	Save acknowledgement and original-to-NUERS ID mapping atomically.
Duplicate response	Treat an identical accepted duplicate as delivered. It is not another sale or another eligible charge.
Unexpected response	Do not mark delivered on an HTML login page, redirect, malformed JSON or mismatched identity.

Storage acknowledgement
durably_recorded confirms that the transaction has been saved in NUERS. It does not mean the BIR has accepted it. Display NUERS Success. Government-status fields are not part of this merchant response.

05 / RESPONSES

Look up and reconcile a transaction

```
curl --silent --show-error --fail-with-body \
  "${NUERS_API_BASE}/${NUERS_INTAKE_ID}" \
  --header "X-NUERS-API-Key: ${NUERS_API_KEY}" \
  --header "Accept: application/json"
```

NUERS_INTAKE_ID is the response field id. A successful lookup returns the stored acknowledgement and current customer email delivery state. The returned receipt_id is a different identifier. Store both under clear column names.

Scenario	Action
POST succeeded, response saved	Use GET /{id} for later checks. Avoid repeatedly resubmitting a delivered event.
POST timed out; no id received	Repeat the exact saved event with the same key context. Duplicate protection returns the existing record if it committed.
GET returns 404	Check id, environment, merchant and assigned branch. Do not assume the original sale is absent and generate a new identity.
Customer email pending	Keep NUERS Success once recorded. Poll customer_receipt separately; email is asynchronous.
Dashboard and ERP differ	Compare environment, original number, event identity and returned IDs, then use request logs and a support ticket.

Source-profile date coverage

The automatic connection covers new originals from its effective creation time while the registered seller details remain compatible. Use the actual original occurred_at. Earlier documents or a historic change in seller registration need a suitable historical-source process through support.

> Accepted originals stay immutable
> Do not modify an accepted event and resend it under the same identity. Do not change the receipt number or event ID to bypass a conflict. Preserve the original and request the supported correction process.

05 / WALLET

Check pricing and available balance

GET /wallet returns the current connection context and published pricing. Read the policy instead of hard-coding PHP 3 forever. Values named *_centavos are integer centavos: 300 means PHP 3.00.

```
{
  "environment": "live",
  "currency": "PHP",
  "available_centavos": 100000,
  "pricing": {
    "fee_centavos": 300,
    "minimum_transaction_centavos": 0,
    "revision": 2
  }
}
```

Abbreviated illustrative response. Use the actual pricing values returned by your wallet endpoint. Super Admin can set a positive minimum: a PHP 500 minimum exempts sales below PHP 500. A PHP 0 minimum charges every primary sale. Changes apply to new acceptances; earlier charges and unchanged retries keep their original result.

Default: PHP 3 per sale, minimum PHP 0	Fee
Primary sale gross PHP 0.01	PHP 3.00
Primary sale gross PHP 499.99	PHP 3.00
Primary sale gross PHP 500.00	PHP 3.00
Identical retry of an accepted sale	No second charge
Supplementary payment receipt	No second primary-sale fee
Sandbox submission	No debit; simulated pricing only

> Insufficient balance: HTTP 402
> An eligible Live write is rejected if the wallet cannot cover its charge. Fund the wallet, then resend the same event unchanged. The rejected attempt does not commit a new transaction or fee.

A balance lookup is advisory. Another concurrent transaction may spend the balance before your POST. NUERS checks and charges during acceptance. The fee uses the final gross before withholding; withholding does not reduce the amount used for a configured minimum.

05 / BATCH

Send up to 50 events atomically

POST to https://nuers.net/api/integrations/transactions/batch with a JSON object containing events. All events must match the key's assigned business and branch.

```
{
  "events": [
    { "...": "complete event A from page 11" },
    { "...": "complete event B with its own stable IDs" }
  ]
}
```

The placeholders above show the wrapper only; they are not valid payload fields. Replace each object with a complete event. An accepted batch returns HTTP 201 and this response shape:

```
{
  "atomic": true,
  "accepted": 2,
  "results": [
    { "...": "acceptance response for event A" },
    { "...": "acceptance response for event B" }
  ]
}
```

01 Queue complete immutable events first. Group 1-50 with the same environment and assigned branch.

02 Validate each result against its original source_event_id and external_reference; save each returned id. Do not rely only on the top-level accepted count.

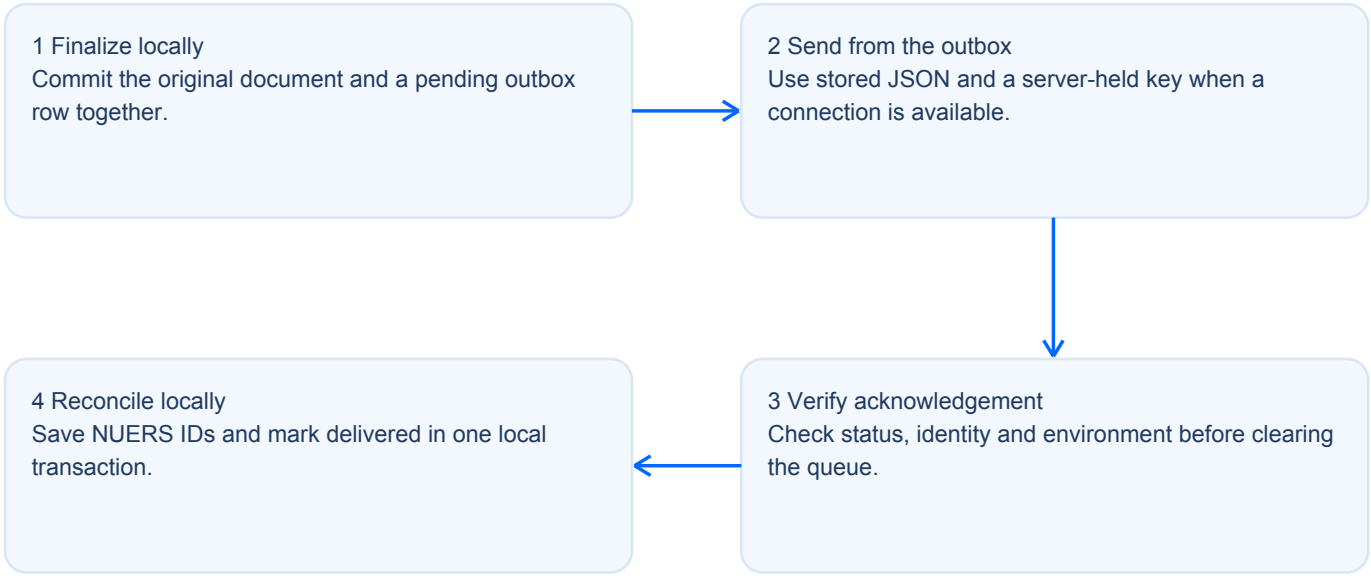
03 If any new event fails validation or the wallet cannot cover the batch, the new writes and debits roll back. Previously accepted events still exist.

04 For an uncertain timeout, resend the same batch or its unchanged events. Existing accepted events return duplicates; do not generate replacements.

> Count and rate limits
> accepted counts result entries, including duplicates; it is not a count of new wallet charges. Batch requests also have a 10-per-minute route limit, in addition to key and shared limits. Honor HTTP 429 and Retry-After.

06 / RELIABILITY

Deliver sales after an internet outage



Minimum durable outbox fields

Field group	Store
Identity	environment, source_system, source_event_id, external_reference, receipt_number
Original event	Exact finalized JSON, original occurred_at and a local content hash
Delivery state	pending / sending / delivered / needs_review, attempt count, next attempt time
Acknowledgement	NUERS id, transaction_id, receipt_id, accepted_at, response environment
Operations	Redacted last error, timestamps and worker-lock / lease information

A timeout is an unknown result
The server may have committed before the connection was lost. Leave the event pending and retry its original content. Never create a new event ID just because the first request timed out.

NUERS cannot pull transactions from an offline ERP by itself. Your connector or a merchant-operated manual retry sends the retained originals after connectivity returns.

06 / RELIABILITY

Duplicate protection and retry policy

NUERS checks stable source identity within the merchant, branch and environment. Keep source_system, source_event_id and the family-specific external_reference unchanged. The original document number is also protected.

Result	What to do
201 accepted	Commit acknowledgement; stop retries for this event.
200 + duplicate: true	Same original already accepted. Save the existing IDs; stop.
Network failure / timeout / 5xx	Keep pending. Retry the identical event using backoff and jitter.
429	Wait according to Retry-After when provided, then retry unchanged.
402	Pause eligible Live sending, fund the wallet and retry unchanged.
401 / 403	Pause and fix the credential, scope, IP, expiry or account issue.
409	Stop automatic retries. Review identity/content or profile mismatch.
422	Inspect field errors. Correct unaccepted mapping data only after checking whether an earlier attempt was accepted.

Suggested connector backoff

For transient failures, start around 2 seconds and increase exponentially with random jitter, capped at 5 minutes. Honor a longer server Retry-After. Alert an operator after a sustained failure window; keep the pending row instead of dropping it.

> Preserve the accepted event
> Do not replace occurred_at with the retry time, reorder items, round prices differently, change JSON strings to numbers, add nulls or regenerate IDs. Object-key order is normalized by NUERS, but payload values and types remain significant.

Retries require a currently authorized key. Key rotation does not reset the identity of a sale. Never bypass a conflict by inventing a new receipt_number or external_reference.

06 / RELIABILITY

Merchant-operated manual retry

Use this when the ERP has exported pending events but its automated sender is unavailable. It reuses the same transaction contract and duplicate checks.

01 Export the finalized pending events from your system as JSON. Use an array of events or an object with an events array. Keep all original identifiers, amounts, timestamps and source context.

02 Open Merchant > API Integration > Manual Retry. Select the matching Live or Sandbox environment.

03 Import a file of at most 50 events and 2 MB. This JSON limit is separate from Business Profile document-upload limits.

04 Protect the browser queue with a passphrase of at least 16 characters. Use a private workstation and your merchant-owned key with read and write scopes.

05 Review and send the queue. Branch and environment must match the key. Keep entries pending until their response confirms durable acceptance.

06 Reconcile successful responses back to the ERP and mark those original outbox rows delivered. Store the returned NUERS IDs.

> Browser storage is a recovery aid
> Keep the authoritative pending queue in your source system. Browser data can be cleared, and the API key is kept only in memory. Do not use the browser as the only copy of unsent transactions.

After reconnecting the automated sender

It may encounter an event already sent manually. Resend its original payload if its local acknowledgement is missing; duplicate protection returns the existing record. Reconcile that record and remove the pending state locally.

If an event is rejected, preserve the error, original JSON and identity. Use the same review steps as the automated connector. Do not re-export the sale under a new identity just to make the queue empty.

07 / WEBHOOKS

Connect your HTTPS receiver

Webhooks notify your system after an event is accepted. They complement status polling; they do not replace the durable acknowledgement returned by POST.

01 Prepare an HTTPS callback route on a public domain that your organization controls. Use port 443. Keep Live and Sandbox receiver configuration separate.

02 Open API Integration > Webhooks. Add a name, callback URL, environment and event subscriptions. Confirm ownership of the destination.

03 Save the one-time webhook signing secret in the receiver's secret manager. This is separate from your NUERS API key. Record its key version.

04 Deploy the signature verification and durable inbox described on pages 29-30. Configure the expected merchant_id and environment.

05 Choose Verify Receiver. Your route must verify the signed challenge and echo its nonce. Verification leaves delivery paused.

06 Choose Enable, then send a test event and inspect the delivery result. The test event is synthetic and does not create a sale.

> Destination restrictions
> Use a public HTTPS hostname on port 443, without credentials, query parameters or fragments. Private/local addresses, IP literals and redirects are rejected. Expose the actual receiver route directly.

A callback URL cannot be generated for your ERP by NUERS because the endpoint must exist in your own system. If the receiver is not ready, leave callbacks paused and use GET /{id} for reconciliation.

07 / WEBHOOKS

Choose events and process the envelope

Event type	Meaning
transaction.accepted	A transaction was durably accepted. Recommended general subscription for a source connector.
source.received	An original source document was captured.
invoice.issued	A new original was issued through the separate NUERS issuance flow.
payment.recorded	A payment event was recorded through its applicable flow.
webhook.test	Synthetic receiver test; do not book a sale.
webhook.endpoint_challenge	Receiver verification challenge; echo the signed nonce.

```
{
  "id": "44444444-4444-4444-8444-444444444444",
  "type": "transaction.accepted",
  "environment": "sandbox",
  "merchant_id": "<your merchant UUID>",
  "subject_id": "<accepted intake UUID>",
  "occurred_at": "2026-10-18T10:00:03+08:00",
  "data": { "id": "<accepted intake UUID>",
            "status": "durably_recorded" }
}
```

Abbreviated envelope. data includes acceptance metadata; your handler should tolerate additional fields. It does not carry the API key or a full line-item/customer payload. Fetch details using the accepted intake id and your configured API base when needed.

> Avoid counting a sale twice
> transaction.accepted and source.received can describe the same source capture. Subscribe to one for your business action, or deduplicate that action by the accepted subject / intake id as well as the delivery event id.

07 / WEBHOOKS

Verify every signed request

Header	Value
X-NUERS-Event	Webhook event UUID.
X-NUERS-Timestamp	Unix timestamp in seconds.
X-NUERS-Signature	v1= followed by a 64-character SHA-256 HMAC hex value.
X-NUERS-Environment	live or sandbox.
X-NUERS-Key-Version	Signing-secret version configured for this receiver.
Content-Type	application/json.

Signature input

```
message = timestamp + "." + event_id + "." + raw_body
expected = HMAC_SHA256_HEX(signing_secret, message)
header   = "v1=" + expected
```

01 Read the exact raw request bytes once. Reject duplicate security headers, malformed signatures and timestamps more than 300 seconds from your synchronized server clock.

02 Compute the HMAC before parsing JSON. Compare it in constant time. Do not reserialize JSON, trim the body or normalize whitespace before signing.

03 After signature verification, parse JSON and require matching event id, expected environment and merchant_id. Check key version against the receiver configuration.

04 For a challenge, respond HTTP 200 with {"nuers_challenge": "received nonce"}. For business events, commit a durable inbox row before returning 2xx.

Challenge nonces are 48 characters and expire after 10 minutes. Start a new verification if expired. Do not reuse challenge handlers as a general unauthenticated echo endpoint.

07 / RECEIVER CODE

PHP signature-verification core

Call this before your business logic. The HTTP adapter must reject duplicate security headers and supply lower-case header names, the raw body and private receiver settings.

```php
<?php
function verifyNuers(string $raw, array $h, array $cfg): array
{
    $id = $h['x-nuers-event'] ?? '';
    $ts = $h['x-nuers-timestamp'] ?? '';
    $sig = $h['x-nuers-signature'] ?? '';
    if (strlen($raw) > 65536 ||
        !preg_match('/^[a-f0-9-]{36}$/D', $id) ||
        !ctype_digit($ts) || strlen($ts) > 12 ||
        abs(time() - (int)$ts) > 300 ||
        !preg_match('/^v1=[a-f0-9]{64}$/D', $sig) ||
        ($h['x-nuers-environment'] ?? '') !== $cfg['environment'] ||
        ($h['x-nuers-key-version'] ?? '') !== $cfg['key_version']) {
        throw new RuntimeException('Invalid webhook headers');
    }
    $mac = hash_hmac('sha256', $ts.'.'.$id.'.'.$raw, $cfg['secret']);
    if (!hash_equals($mac, substr($sig, 3))) {
        throw new RuntimeException('Invalid webhook signature');
    }
    $event = json_decode($raw, true, 32, JSON_THROW_ON_ERROR);
    if (($event['id'] ?? null) !== $id ||
        ($event['environment'] ?? null) !== $cfg['environment'] ||
        ($event['merchant_id'] ?? null) !== $cfg['merchant_id']) {
        throw new RuntimeException('Unexpected webhook identity');
    }
    return $event;
}
```

cfg contains secret, key_version as a string, environment and merchant_id. Map verification failures to HTTP 401; do not process the body. Validate the event type, handle signed challenges, then apply the inbox rules on page 31.

> Complete the HTTP route
> Read php://input once, verifyNuers first, then handle the signed challenge or commit the business event to your inbox. Return JSON with Content-Type: application/json. Do not return 2xx before persistence succeeds.

07 / RELIABILITY

Webhook inbox, retries and rotation

Acknowledge only after durable storage

01 Begin a local database transaction and insert the verified raw event under a unique (environment, event_id) key. Store its body hash and pending processing state.

02 If the key already exists and the hash matches, return the existing successful acknowledgement. If the content differs, reject it and alert an operator.

03 Commit before returning HTTP 200 or another 2xx. If persistence is unavailable, return 503 so delivery can retry.

04 Process the inbox asynchronously. Upsert the existing source-to-NUERS mapping; do not create a second ERP sale. Events can arrive out of order.

Delivery behavior	Receiver / merchant action
Timeout	NUERS waits up to 8 seconds. Keep acknowledgement fast and process asynchronously.
Transient failures	Automatic retry uses backoff and jitter, up to 12 attempts within 24 hours.
Non-retry responses	3xx and 400, 401, 403, 404, 405, 410, 413 or 422 stop automatic retries. Fix the cause before manual retry.
Manual retry	Up to three retry cycles. Event id and body stay stable; deduplicate them.
Pause / revoke / change	Pending deliveries incompatible with the new policy are cancelled. An already-in-flight request can still arrive.
Secret rotation	Rotation pauses delivery. Deploy the new secret/version, verify again, then enable.

Recover from missed callbacks
Compare the source outbox, accepted intake IDs and merchant records. Poll GET /{id} when delivery is delayed or a callback is unavailable. Callback delivery status does not change whether the original POST was accepted.

08 / TROUBLESHOOTING

HTTP status and recovery reference

Status	Likely cause / safe next step
200	Read succeeded, or identical POST duplicate. Check the full response.
201	New event / accepted batch. Validate each acknowledgement before clearing the queue.
401	Missing or invalid key, or conflicting credentials. Load the correct secret; never paste it into a ticket.
402	Insufficient wallet balance. Fund wallet and resend unchanged.
403	Expired/revoked key, scope, egress IP, inactive account or assigned-branch mismatch. Fix the stated policy issue.
404	Unknown record or wrong merchant, branch or environment. Confirm the returned intake id.
409	Source identity/content conflict, duplicate original number or source-profile mismatch. Stop and investigate.
413	Request exceeds an edge/application limit. Reduce batch size; preserve each event unchanged.
415	Send Content-Type: application/json.
422	Invalid fields, totals, tax treatment or benefit evidence. Read message and errors. Correct the unaccepted mapping carefully.
429	Quota or rate limit. Honor Retry-After; slow the connector.
5xx / network timeout	Outcome may be unknown. Retain the event and retry unchanged with backoff.

Typical field-validation response

```
{
  "message": "The given data was invalid.",
  "errors": { "items.0.quantity": ["Validation detail"] }
}
```

Illustrative error structure. Some failures return only message; an edge failure can be non-JSON. Keep HTTP status and a redacted diagnostic, not raw credentials or beneficiary details.

08 / SECURITY

Protect the connection in production

Area	Required connector practice
Credentials	Server-side secret manager, least scopes, restricted access, expiry reminders and controlled rotation.
Network	HTTPS verification enabled; stable public egress allowlist; no redirect-following with credentials.
Tenant / environment	One configured merchant context per connector; separate Live/Sandbox secrets, queues and webhook inboxes.
Payload integrity	Persist immutable source JSON; use stable original IDs; restrict changes after finalization.
Privacy	Protect customer and beneficiary details in databases, backups and exports. Use evidence references instead of attachments.
Logs	Record correlation IDs, timestamps, status and redacted field errors. Never log API keys, signing secrets or full identity evidence.
Webhook ingress	HMAC on raw bytes, timestamp window, duplicate-header rejection, durable replay protection and synchronized clocks.
Operations	Monitor queue age, wallet blocks, expiry, repeated validation errors and webhook failures; assign an owner for alerts.

Recommended operational alerts

Alert when pending Live events exceed your normal delivery window, an account/key returns 401 or 403, wallet rejections persist, or a batch repeatedly fails. Track oldest pending age and reconciliation mismatches, not only request counts.

> If a credential is exposed
> Revoke it in API Integration, create a replacement and update your connector securely. Review request logs and affected time ranges. Use stable event identities when resuming pending traffic.

09 / LAUNCH CHECKLIST

Prove the connection before going Live

Test	Expected evidence
Authentication	Wallet lookup succeeds for the intended merchant and environment; wrong key is rejected.
Single capture	Synthetic Sandbox sale returns 201 and expected tax amounts.
Identical retry	Same event returns 200 duplicate with the same accepted IDs.
Conflicting retry	Changed content under the same identity is rejected with 409.
Tax / benefits	Your supported product treatments and eligible categories reconcile to the original amounts.
Network loss	A simulated lost response leaves the queue pending; resend resolves without a second sale.
Batch	Valid batch acknowledges each event; one invalid event does not commit other new events.
Environment boundary	Sandbox records and acknowledgements cannot clear Live rows or debit the wallet.
Webhook, if used	Challenge passes; test event commits once; invalid signatures fail; duplicate delivery causes no second business action.
Operations	Expiry alerts, wallet funding, backups and a named support owner are ready.

Controlled Live cutover

01 Use the assigned Live source configuration and a Live key. Confirm account status, current pricing, sufficient balance and the original-date coverage.

02 Send one authorized actual finalized business event. Do not turn the printed sample sale into a Live record.

03 Reconcile the original, returned IDs, calculated amounts and wallet entry. Then enable normal traffic and monitor the first operating period.

09 / OPERATIONS

Reconcile records and request support

Merchant dashboard checks

Use API Integration > Usage & Logs for request outcomes and Webhooks for delivery status. Select the same environment as your connector. Compare accepted originals in Invoices & Receipts, and reconcile eligible charges in Wallet.

Use Sales Reports to filter the relevant date range and sales category, including senior citizen, PWD, national athlete/coach and solo parent. Keep the same filters and environment when exporting so totals match your ERP reconciliation.

Daily comparison	What should match
Original identity	Source system, event ID, external reference, number and original date.
Document amounts	Gross, VAT buckets, withholding and net payable.
Special sales	sale_category, statutory_discount_amount and vat_relief_amount.
Delivery	Every finalized outbox row is either acknowledged or visibly pending / under review.
Wallet	Eligible newly accepted Live sales have the expected policy charge; duplicates have no second charge.

Send a useful support request

Open Merchant > Submit a Request. Include your business name, environment, endpoint/method, timestamp with timezone, HTTP status, redacted error message, source_event_id, external_reference and returned intake id if available. Describe whether the issue is a first attempt or a retry.

> Keep the report safe
> Never include the API key, webhook signing secret, MFA code or full beneficiary identification. Share a redacted payload only when requested through the authorized support process. Preserve the original privately for reconciliation.

QUICK REFERENCE

Connection essentials at a glance

Item	Value / action
Base URL	https://nuers.net/api/integrations/transactions
Authentication	X-NUERS-API-Key; HTTPS; server-side use only
Schema	NUERS-ingestion-1
Mode for ERP originals	source_capture, explicitly supplied
Setup	Active account; name + environment; automatic assigned business/source context
Primary document	sales_invoice + invoice.finalized
Supplementary payment	payment_receipt + payment.recorded
Payload limit	1-100 items/event; batch 1-50 events
Acknowledgement	201 new, or 200 with duplicate: true; status durably_recorded; verify environment and IDs
Retry identity	Same original JSON, source IDs, reference, number and occurred_at
Wallet rejection	402: fund and resend unchanged
Callback security	HMAC-SHA256 of timestamp.event_id.raw_body; verify before parsing
Support	Merchant dashboard > Submit a Request

Manual discounts and version

Edition 1.4, issued 29 October 2026 for contract NUERS-ingestion-1. Manual amount/percentage discounts are detailed on pages 37-38. This guide covers connecting a merchant-owned system and saving its already-issued originals. Separate original issuance, historical-source review and BIR reporting controls are outside this connection workflow.

Use the current values shown in the merchant dashboard for credentials, source identifiers, expiry, limits and pricing. Keep this guide with your connector implementation and retest its acceptance suite when adopting a later contract version.

10 / MANUAL DISCOUNTS

Amount or percentage, invoice or line

Send original unit prices before discount. Add the following object at the top level to discount the whole invoice, or inside one items[] entry to discount that line only.

```
"manual_discount": {
  "type": "amount",
  "value": "112.00",
  "reason": "Loyalty discount"
}
```

Field	Accepted input
type	amount for a fixed PHP amount, or percentage for 0-100%.
value	Non-negative; at most two decimals. Use a two-decimal string. Amount cannot exceed the original line/document price.
reason	Required non-blank text, maximum 500 characters. Record the actual discount reason.

One VAT-inclusive item: PHP1,120.00

Discount method	Expected result
amount: 112.00	Manual discount 112.00; VATable base 900.00; VAT 108.00; gross_amount 1008.00.
percentage: 10.00	Same result. Existing items[].discount_pct: 10 also remains supported.

With vat_inclusive=true, discount amounts include VAT. With false, they reduce the pre-VAT price before VAT is added. A fixed line discount applies to the whole quantity extension, not separately to every unit. VAT follows the recorded tax type of each discounted line.

> Keep totals consistent
> gross_amount is the final discounted total including VAT and before withholding. Do not pre-discount unit_price and also send manual_discount. Invalid values or inconsistent totals return HTTP 422 before financial records are committed.

10 / MANUAL DISCOUNTS

Allocation, reconciliation and reporting

01 Choose either one document discount or line discounts. A document discount cannot be combined with any line discount or sales_benefit.

02 Document discounts allocate proportionally by each original line extension. NUERS uses integer centavos and distributes leftover centavos to the largest fractional shares; original line order breaks ties. Use per-line fixed amounts if your ERP uses a different allocation.

03 A line manual_discount cannot be combined with non-zero discount_pct or a statutory-eligible line. A basket may have statutory-eligible items and separate non-eligible lines with manual discounts.

04 Compare calculation.manual_discount_amount, discount_amount (manual plus statutory), statutory_discount_amount, VAT buckets and gross_amount to the original invoice. Gross totals already contain the reduction; do not subtract it again.

Where	What is recorded
API response / source record	Manual and statutory totals separately; line discount_reason plus discount_pct or fixed_discount_amount in PHP. fixed_discount_amount is a response field, not a request input.
Merchant and admin records	Manual Discount is shown separately from Statutory Discount and VAT Adjustment. Filtered CSV, sales XLSX/PDF and document copies retain the breakdown.
Sandbox	Same validation and discount calculation; test records stay isolated and do not debit the Live wallet.
Wallet / retries	Fee eligibility uses final discounted gross. Identical retries have no second charge; changing discount or reason under an accepted identity returns 409.

Preserve the original
These fields record an ordinary discount reflected at sale time. Later conditional rebates require the appropriate correction workflow. Existing accepted documents are not silently changed to add missing discount details.

Tax reference: BIR RR 16-2005, section 4.106-9, addresses invoiced at-sale discounts for goods. Product eligibility and the applicable tax basis remain the responsibility of the merchant. BIR published regulation.

11 / CONNECTOR SALES DASHBOARD

Show a truthful NUERS status

Every ERP, POS or e-commerce sales row must show NUERS Status, failure reason, environment, NUERS reference and last attempt. Show Success only after a valid 201 or 200 duplicate acknowledgement with matching source_event_id, external_reference and environment, non-empty id/transaction_id/receipt_id, status=durably_recorded and nuers_status=success. Persist these identifiers atomically with the local outbox acknowledgement.

Dashboard state	When to display it
Pending	The local sale is saved but has not yet received a valid NUERS acknowledgement.
Success	Matching acknowledgement with nuers_status=success and status=durably_recorded. Store all returned NUERS IDs.
Error + reason	A known rejection. Show error.reason and relevant field messages. Keep the source sale for correction or funding.
Pending confirmation	Timeout, malformed response or error.outcome=unknown. Retry the identical event; never create a replacement identity.

Show Error plus error.reason and applicable errors field messages for a rejected request. Keep the original sale locally. 402 requires wallet funding; 401/403 require credential or permission correction; 409 requires reconciliation; 422 requires mapping correction. For timeouts, invalid JSON, malformed 2xx or error.outcome=unknown, show Pending confirmation and retry the identical original event. Honor Retry-After and bounded backoff for 429/5xx. Never render server messages as HTML or expose credentials to the customer.

> NUERS status in the sales row
> Merchant transaction, batch and wallet responses do not expose bir_acceptance, bir_connection, bir_status or internal preparation blockers. Display NUERS status in the connector. NUERS Success confirms platform recording; it does not assert government acceptance.

Download the small connector UI adapter from https://nuers.net/guides/NUERS-Transaction-Status.mjs. Use it in your backend with the saved source event and configured environment. Render messages as plain text. Your ERP/POS developer must connect these fields to its own sales dashboard.

11 / API RESPONSE CONTRACT

Success and error examples

Selected fields; all original identifiers, calculated amounts and wallet details remain available. The event is recorded before the success response is returned.

```
{
  "nuers_status": "success", "status": "durably_recorded",
  "environment": "live", "duplicate": false,
  "id": "<inbox-id>", "transaction_id": "<transaction-id>",
  "receipt_id": "<receipt-id>",
  "source_event_id": "erpA:invoice:0001",
  "external_reference": "erpA:INV-0001", "error": null,
  "customer_receipt": {
    "delivery_id": "<delivery-id>", "status": "queued",
    "reason": "Customer receipt is queued for email processing."
  }
}
```

Rejected request: HTTP 422

```
{
  "nuers_status": "error", "code": "VALIDATION_FAILED",
  "message": "The customer email is invalid.",
  "error": {
    "code": "VALIDATION_FAILED",
    "reason": "The customer email is invalid.",
    "retryable": false, "outcome": "rejected"
  },
  "errors": {"customer_email": ["Use one valid email address."]}
}
```

Actual wording follows the validator. Branch on HTTP status and machine code, never on English message text. retryable=true means a bounded unchanged retry is appropriate; 402 requires funding first. For atomic batches, use results[] only after the entire batch succeeds.

12 / CUSTOMER E-RECEIPTS

Map checkout email and saved branding

```
"customer_name": "SAMPLE Customer",
"customer_email": "customer@example.test"
```

Optional customer_email: one valid checkout email address, maximum 254 characters. Omit or send null when absent. Map the checkout email explicitly; NUERS does not scan arbitrary payload fields. Invalid addresses return 422 before acceptance. Never use a customer address as an API credential.

A newly accepted Live event containing customer_email queues one customer PDF email in the same database transaction. Exact retries return the same record and do not queue another email or fee. Failed/rolled-back events queue no email. Sandbox always returns suppressed_sandbox and never emails customers. Existing transactions are not emailed retroactively.

Use the merchant receipt design

Email attachments use the same receipt renderer and E-Receipt Logo saved under Merchant Invoice Settings → Logo and Branding. This logo is independent from the Business Profile Picture; both uploads save immediately without administrator approval. Original seller identity, lines, taxes, discounts and totals remain the recorded snapshot. Save branding before the email worker renders the PDF; changes affect subsequent renders and cannot recall an already sent attachment.

01 Merchant opens Invoice Settings, saves the dedicated E-Receipt Logo, colors, layout, footer and receipt preferences.

02 Connector commits the customer email with the finalized source payload and sends it with the same immutable event identity.

03 NUERS commits the sale and encrypted email job together, then returns Success plus customer_receipt.status=queued.

04 The email worker renders the same merchant document as a PDF attachment and sends it through configured platform SMTP.

> One copy per accepted event
> Keep customer_email unchanged on an identical retry. Changing it under an already accepted source identity returns 409. Use the merchant document Send Copy action for a separately requested copy; do not create another sale.

12 / CUSTOMER EMAIL OPERATIONS

Track delivery independently

customer_receipt.status	Meaning
not_requested	No checkout email supplied.
suppressed_sandbox	No customer email is sent in Sandbox.
queued / preparing / sending	Asynchronous work is waiting or in progress.
smtp_accepted	SMTP accepted the message; inbox delivery is not confirmed.
failed / expired / cancelled	Mail preparation, queue lifetime or account/document eligibility prevented sending.
uncertain	SMTP or worker outcome is unknown. Investigate before a manual resend.

Poll GET status_url with the same environment and transactions:read scope for customer_receipt.status and reason. States: not_requested, suppressed_sandbox, queued, preparing, sending, smtp_accepted, failed, uncertain, expired, cancelled. smtp_accepted means the SMTP server accepted the message, not confirmed inbox delivery. Email failures never change a successfully recorded sale to Error. Do not retry the sale to resend mail; use the merchant document Send Copy action after investigating an uncertain delivery.

Connector acceptance checklist

01 Show NUERS Status, environment, reference, last attempt and the error reason on every sales row.

02 Use synthetic addresses and separate Sandbox keys during mapping tests. Sandbox will not send test mail.

03 Test missing/invalid email, retry deduplication, batch rollback and branded PDF output in isolated fixtures.

04 Confirm the production email worker and SMTP configuration. Monitor delivery history; do not claim delivered when only smtp_accepted is known.

Use the public API reference, this PDF and the merchant connection guide from the same release: 2026-10-29.invoice-documents. Existing records are not emailed retroactively.

13 / WITHHOLDING & CORRECTIONS

Buyer-confirmed CWT

For buyer-confirmed CWT, send withholding_tax in PHP and withholding_basis_reference. Send withholding_details with withholding_agent=true, classification, atc (two uppercase letters and three digits), rate (percentage), base, amount and evidence_reference. Decimal money fields have at most two decimals. amount must equal withholding_tax and rounded base × rate / 100. CWT reduces net_payable; gross_amount, sales, VAT and the wallet fee basis stay unchanged. Existing amount-and-basis integrations remain supported; use the structured buyer confirmation for new mappings.

```
{
  "withholding_tax": "10.00",
  "withholding_basis_reference": "BUYER-CONFIRMATION-001",
  "withholding_details": {
    "withholding_agent": true,
    "classification": "Buyer-confirmed withholding agent",
    "atc": "WC160",
    "rate": "1.00",
    "base": "1000.00",
    "amount": "10.00",
    "evidence_reference": "BUYER-CONFIRMATION-001"
  }
}
```

Example only
ATC and rate are illustrative. The buyer/payor must determine the applicable classification and rate. A PHP 1,120 invoice with PHP 10 CWT still records PHP 1,120 gross and PHP 120 VAT; net payable is PHP 1,110.

13 / WITHHOLDING & CORRECTIONS

Preserve the original

For CWT supplied or corrected after issuance, open Merchant → Compliance Operations → CWT adjustment memos. Select the issued original and record buyer/payor confirmation. Issuing a registered credit memo requires current authority and a credit-memo series. The original stays unchanged; sales and VAT adjustments are zero. Subsequent corrections reference the current CWT amount, with stale drafts rejected.

For an increase, issue an additional electronic invoice referencing the original; for a decrease, use a credit memo. The native merchant form does not create debit memos for new adjustments. Original records remain immutable. The intake API does not accept credit-memo document types; use the controlled merchant workflow for a NUERS-issued adjustment.

01 Open Compliance Operations and choose CWT adjustment memos.

02 Select the original invoice; provide buyer-confirmed classification, ATC, rate, base, amount and evidence.

03 Review the draft. Official issuance requires verified authority and an active credit-memo series.

04 Download the issued memo and retain the original invoice. Later corrections use the latest issued CWT adjustment.

14 / OUTAGE RECOVERY

Authorized manual invoices

Record an outage and its authorized manual invoices under Merchant → Compliance Operations, then record verified restoration. On a replacement Live invoice, send manual_recovery_id from that register and the original manual number in manual_invoice_number. The replacement must match merchant, branch, buyer and gross amount, explicitly reference the manual invoice and be dated after restoration. One manual invoice links to one replacement. A mismatch rolls back the new event and fee. Sandbox cannot link Live recovery records. The register tracks the three-calendar-day recovery deadline; it does not issue manual authority or claim BIR transmission.

```
{
  "manual_recovery_id": "00000000-0000-4000-8000-000000000002",
  "manual_invoice_number": "SAMPLE-MANUAL-001"
}
```

01 Record the outage and retain its evidence.

02 Register each authorized manual invoice with its original number, branch, buyer, date, gross amount and authority reference.

03 Record verified service restoration. Track the replacement deadline shown in the register.

04 Send the replacement from your owned system with the recovery fields and matching original identity; preserve its acknowledgement.

05 Review unreconciled manual invoices. A retry must use the identical original event; a second replacement is rejected.

15 / PLATFORM CONTROLS

Readiness and fee invoices

Live storage intake remains available while ESP accreditation enforcement is prepared but disabled. This is an operational setting, not a statement of BIR authorization. Official NUERS issuance and BIR transmission retain separate authority checks.

NUERS service-fee statements use posted wallet debits and cannot bill a debit twice. An administrator prepares the billing period. The merchant can review and download the statement under Compliance Operations. Official service invoices require a separate registered NUERS billing entity, verified issuance authority and an active invoice series. Preparing or issuing a fee document does not debit the merchant wallet again.

> External evidence remains separate
> BIR credentials and transmission, actual accreditation, independent security certifications and production disaster-recovery evidence cannot be replaced by local software tests. Your records continue to be available under the current approved operational scope.

16 / INVOICE IDENTITY

Preserve the seller invoice number

Original seller invoice/document number: 1–50 characters; no control characters or surrounding whitespace. Preserve prefixes and leading zeros. This is EIS CompInvoiceId, not the separate 24-character EisUniqueId. Source capture preserves the source number; NUERS issuance uses the next registered series number. Never renumber accepted originals.

Record mode	Numbering behavior
source_capture	Retains the original finalized ERP/POS number. The connector must enforce the source system's registered series; NUERS does not allocate a second invoice number for the copy.
nuers_issuance	Reserves the next number in the configured business / branch / document-family series. Checks its active period, range, recorded authority and collisions. A failed issuance rolls back the reservation.
Existing originals	Never renumber, overwrite or reuse a number, including a voided number. A legal correction has its own document identity and references the original.
Preview / draft	A unique DRAFT reference identifies unsaved work. Only successful issuance allocates a registered serial; a draft is not a tax invoice.

Buyer and related-document mapping

business, retail or walk_in. business requires the buyer registered name, registered address and supported nonzero TIN. B2C address/TIN are optional. Declare the actual buyer type.

Original invoice number for a related payment or additional invoice, maximum 50 characters. Existing originals remain immutable. This reference is printed on the receipt; it does not cancel or replace the original.

> Keep identity stable on retry
> Use the same complete payload, source_event_id and external_reference. After acceptance, changes to buyer, number, registration or financial data conflict with the stored event. Do not invent a new identity to bypass a conflict.

Reference: BIR EIS JSON Format v2.1 (CompInvoiceId); RR 7-2024, Section 6; RMC 98-2026. Official API guide: https://eis-cert.bir.gov.ph/#/apiGuide. BIR-issued credentials and authority remain separate prerequisites.

16 / PRINTED EVIDENCE

Invoice and receipt fields

Optional recorded source registration evidence. system_type: CAS, POS or ELECTRONIC. Supply authentic applicable PTI, PTU/AC, authority issue date, series range and POS MIN/machine/software identifiers. This snapshot is printed on copies; supplying it does not establish BIR approval. Missing historical evidence is shown as not recorded. Native NUERS issuance requires an evidence-backed series.

```
{
  "customer_type": "business",
  "invoice_reference": "SAMPLE-ORIGINAL-000001",
  "document_registration": {
    "system_type": "CAS",
    "pti_number": "SAMPLE-PTI-REFERENCE",
    "ac_number": "SAMPLE-AC-REFERENCE",
    "authority_issued_at": "2026-01-01",
    "series_start": "SAMPLE-000001",
    "series_end": "SAMPLE-999999"
  }
}
```

Synthetic references only. Replace them with authentic recorded evidence; never send example values to claim authority. For POS, also map the MIN and machine serial or software license, plus final PTU. For a non-VAT seller, non_vat_basis distinguishes percentage_tax from an evidenced exempt basis.

Shared preview / email / PDF	Printed content
Seller and buyer	Registered seller name/address, VAT or Non-VAT Reg TIN and five-digit branch; applicable buyer name/address/TIN.
Items and amounts	Description, optional unit, quantity, unit cost, discounts, VAT classifications and separate totals. Buyer-confirmed CWT includes ATC, rate and base; it reduces net payable, not sales/VAT.
Evidence and copy	Recorded authority, date and series; original-invoice reference; REPRINT. Supplementary documents carry the input-tax warning. Unknown evidence stays not recorded.

Design stays consistent with Merchant Invoice Settings. RR 7-2024 specifies invoicing information; RMC 112-2026 addresses CWT presentation. These fields and local validation do not establish BIR approval or acceptance.